

TP Integrador — El buffet de Alberto

Grupos de 4 o 5 · entrega grupal · defensa oral individual

Leé con atención el pedido del cliente. Tu tarea es entender qué necesita y descubrir cómo resolverlo con lo que vimos de archivos. El cliente cuenta su problema a su manera; el “cómo” lo tenés que poner vos.

Hola, ¿qué tal? Soy Alberto, el dueño del buffet Albert Einstein. Abrimos hace unas semanas y, por suerte, viene lleno de gente. El problema soy yo con los papeles: soy un desastre para los números.

Cuando arrancamos, un sobrino me armó un programita que va guardando cada venta en un archivo (me dijo que se llama `comandas_historicas.dat`). El tema es que anota todo de cualquier manera: en cada venta me escribe el nombre completo del mozo, la fecha, el producto y la cantidad, una abajo de la otra y sin ningún orden. Ahora que quiero sacar cuentas, es un despelote y no entiendo nada.

Necesitaría que me armen un sistema en serio. Les cuento lo que preciso, más o menos en el orden que se me ocurre:

- **Primero, tener a mis mozos prolijos:** una lista con cada mozo una sola vez, con un identificador que sea un número propio (así no escribo el nombre largo a cada rato), y saber cuánta comisión lleva juntada cada uno desde que abrimos.
- Poder mirar las ventas día por día: que cada día tenga su propia planilla, que adentro figure el número del mozo en vez del nombre, y que esté ordenada por mozo así los encuentro de una.
- Que se me actualice el stock solo: tengo una lista de productos con lo que me queda de cada cosa (el inventario), pero cada venta me come mercadería y yo nunca la descuento. Quiero que, al procesar las ventas viejas, se me vaya bajando lo que se vendió, sin rehacerme toda la lista de cero.
- Cargar yo las ventas de la semana desde una pantalla con opciones. Me siento a cargar lo de un día: le aviso una sola vez qué día es y, a partir de ahí, todas las ventas que cargue en esa tanda van a la planilla de ese día. Para cada venta, que primero me pida el número del mozo (y si pongo un número que no existe, que no me deje seguir); después el producto y la cantidad; que la comisión la calcule solo según el precio; y que lo guarde en la planilla del día (si ese día todavía no tiene planilla, que la cree). Cuando termino de cargar, que me quede todo ordenado por mozo. Y una cosa importante: quiero que cada mozo entre con su propia clave, así no se cargan ventas entre ellos. Eso sí, me pone nervioso que las claves queden escritas tal cual en el archivo, a la vista de cualquiera que lo abra: me gustaría que se guarden de una forma que no se lean así nomás, aunque sea algo sencillito.
- Juntar la semana en un solo lugar: al terminar la semana, unir todos los días en una única planilla semanal, también ordenada por mozo, para no andar con un archivo por cada día dando vueltas.
- Y lo que más me importa, el resumen del cierre: que me diga, por cada mozo, cuántos productos vendió en la semana y cuánta plata de comisión le tengo que pagar. Y bien abajo, el total de productos que vendió el buffet en toda la semana.

Con eso soy feliz. Sé que suena a mucho, pero para ustedes que saben de esto debe ser pan comido. ¡Gracias, cracks!

— Alberto

Ficha técnica (esto ya está armado)

El programita viejo guarda cada venta con este formato exacto, así que tenés que leer `comandas_historicas.dat` respetándolo tal cual:

```
struct ComandaHistorica {
    char  fecha[11];           // "DD-MM-AAAA"
    char  nombreMozo[50];      // el nombre completo, repetido en cada venta
    int   codigoProducto;     int cantidad;   float comision;
};
```

El inventario ya existe (`inventario.dat`) y está ordenado por código, pero faltan códigos en el medio (no son seguidos):

```
struct Producto {
    int    codigo;   char descripcion[50];   float precio;   int stockActual;
};
```

Vas a tener que crear archivos nuevos. Como mínimo, de cada mozo hay que guardar su número, su nombre y la comisión acumulada; y de cada venta, el número del mozo, el producto, la cantidad y su comisión. Te dejamos una struct sugerida para cada uno:

```
struct Mozo {      int idMozo; char nombre[50]; char password[20]; float totalComision; };
struct Comanda {  int idMozo; int codigoProducto; int cantidad; float comision; };

const float TASA_COMISION = 0.10f;    // la comision de cada venta es el 10% de lo vendido
```

Nombres de archivo a respetar: mozos.dat · comandas_dd-mm-aaaa.dat (uno por día) · comandas_semana_sX-mm.dat (uno por semana) · inventario.dat.

La solución se arma en 4 programas

No hagan un solo programón. El trabajo se divide en cuatro programas, cada uno en su propio .cpp con su propio main (no hay un main.cpp aparte). Cada uno se corre por separado y va dejando los archivos que usa el siguiente. Así, además, cada integrante puede hacerse cargo de un programa.

Programa	Qué hace	Lee → Genera
normalizacion.cpp	Ordena lo que dejó el sistema viejo: arma la lista de mozos numerados con su comisión, separa las ventas por día (con el número de mozo y ordenadas por mozo) y deja el stock al día.	comandas_historicas.dat, inventario.dat → mozos.dat, comandas_dd-mm-aaaa.dat (varios), inventario.dat
ventas.cpp	Pide la fecha del día una sola vez al empezar y abre (o crea) la planilla de ese día. Después, por cada venta: pide el mozo y su clave y los valida, busca el precio, calcula la comisión y agrega la venta a la planilla del día. Al terminar, la deja ordenada por mozo.	mosos.dat, inventario.dat → comandas_dd-mm-aaaa.dat (crea/agrega), inventario.dat
cierre.cpp	Junta las planillas de todos los días de la semana en una sola planilla semanal, ordenada por mozo.	comandas_dd-mm-aaaa.dat (varios) → comandas_semana_sX-mm.dat
resumen.cpp	Arma el resumen del cierre: por cada mozo, cuántos productos vendió y cuánta comisión le corresponde; y el total del buffet.	comandas_semana_sX-mm.dat → reporte por pantalla

Reparto recomendado (5 integrantes): normalizacion.cpp es el más grande, va de a dos (uno la lista de mozos, otro las planillas por día y el stock); ventas.cpp, cierre.cpp y resumen.cpp uno cada uno. En grupos de 4, un programa por persona.

Archivos que vas a manejar

Te da la cátedra: comandas_historicas.dat e inventario.dat. **Vas a generar:** mozos.dat (uno), un comandas_dd-mm-aaaa.dat por cada día (varios), y un comandas_semana_sX-mm.dat por cada semana (varios). El inventario.dat no se crea: se va actualizando.

```
/TP_Buffet/
normalizacion.cpp  ventas.cpp  cierre.cpp  resumen.cpp
comandas_historicas.dat  <- lo da la catedra
inventario.dat        <- lo da la catedra (se va actualizando)

-- al correr los programas se van generando --
mosos.dat
comandas_03-06-2025.dat ... (uno por CADA dia)
comandas_semana_s1-06.dat (uno por CADA semana)
```

El login y las claves (obligatorio)

Para cargar ventas, ventas.cpp pide el número de mozo y su clave; si el mozo no existe o la clave no coincide, no deja cargar. **Las claves no se guardan tal cual** en mozos.dat: se guardan pasadas por una cuenta simple, carácter por carácter, que se pueda deshacer y hacer a mano.

Cuenta sugerida (corrimiento): a cada carácter sumale un número fijo K que elija el grupo. Para verificar el login, a la clave que tipea el mozo le hacés la misma cuenta y la comparás con lo guardado.

Ejemplo con K = 5 (clave "sol"):

```
's'(115) + 5 = 120 -> 'x'
'o'(111) + 5 = 116 -> 't'
'l'(108) + 5 = 113 -> 'q'           se guarda: "xtq"
Login: el mozo escribe "sol" -> +5 -> "xtq" -> coincide -> entra.
```

Ojo: “dividir por dos” no sirve, porque pierde información (16 y 17 divididos por 2 dan lo mismo) y después no se puede volver atrás. La cuenta tiene que poder deshacerse; sumar o restar siempre se puede. La clave inicial de cada mozo la deja `normalizacion.cpp` (por ejemplo, su número como texto), ya guardada con la cuenta.

Cómo lo tenés que resolver (reglas de la materia)

- **Todo con archivos binarios** y la API estándar de C: `fopen` / `fread` / `fwrite` / `fseek` / `fclose`.
- **Registros de tamaño fijo:** nunca *string* dentro de una struct que va a disco; siempre `char[N]`.
- **Recorré con while** (`fread(...) == 1`)
- **Elegí la técnica que corresponda en cada caso** (buscar, ordenar, recorrer, combinar, actualizar). Parte del trabajo es darte cuenta de cuál conviene según cómo estén los datos.

Para la defensa oral

En el oral, cada integrante explica y justifica su parte: por qué eligió esa forma de buscar, de ordenar y de guardar, y qué hace su programa en los casos raros. Repártanse el trabajo de manera que cada uno tenga algo propio para defender.

Algunos casos raros que conviene tener resueltos: un mozo cuyo número no existe; una clave que no coincide; un producto que se quedó sin stock; una planilla del día que todavía no está creada; y el resumen del último mozo del cierre (que no tiene otro mozo después).

Sobre las claves, tienen que poder mostrar a mano cómo se guarda, la lógica de encriptación tienen que armarla directamente ustedes sin ninguna librería extra y tienen que explicar cómo funciona y cómo se verifica una clave con su cuenta, y explicar por qué esa cuenta se puede deshacer.

Qué entregar

La entrega es un **repositorio de GitHub**, uno por grupo. En el README tiene que figurar el número de grupo y los integrantes (nombre y usuario de GitHub). El link se entrega en el campus antes de la fecha límite.

El repo tiene que contener:

- Los **4 programas .cpp** (`normalizacion.cpp`, `ventas.cpp`, `cierre.cpp`, `resumen.cpp`), cada uno con su propio `main`. No hay `main.cpp`.
- Un **README** breve: cómo se compila y se corre cada programa, en qué orden, y quién hizo cada parte. También explicar cómo resolvieron las partes claves.
- Un **.gitignore** que deje afuera los ejecutables y los archivos `.dat` (esos se generan al correr; no se versionan).
- **No** subir los `.dat`: la cátedra corre los programas del grupo sobre su propio dataset de prueba.

Sobre los commits: cada integrante tiene que tener **commits propios** a lo largo del trabajo (no vale un único commit final con todo, ni que suba todo una sola persona). El historial es parte de lo que miramos para la defensa individual.